

En esta práctica vas a estudiar la estructura de una app sencilla en Android (con Jetpack Compose) que simula tirar un dado. Al abrir la app, aparece la imagen de un dado y un botón que dice *Roll* (tirar). Cada vez que el usuario pulsa el botón, el programa genera un número aleatorio del 1 al 6, cambia la imagen del dado según ese número y además muestra el valor obtenido en texto (por ejemplo: "Número: 4").

En resumen: es una aplicación de ejemplo para aprender, que enseña cómo manejar estados, mostrar imágenes y texto, y responder a la interacción del usuario con un botón.

En esta actividad partiremos de un código ya realizado. En la unidad 2 veremos con más detalle cómo podemos construir una app con Jetpack Compose desde cero.

Se valorará el nivel de detalle de las explicaciones y capturas de pantalla mostradas en cada apartado. Debe mostrarse cada uno de los pasos realizados en cada apartado.

Se debe entregar este documento relleno y además el proyecto Android en un zip. El código debe incluir las modificaciones realizadas en la práctica (si algún punto no se incluye, este no se valora aunque esté cumplimentado en la memoria).



PASO 1 (1 punto)

Clona el proyecto con Git, para ello:

1. Ve a la carpeta de proyectos de Android Studio

```
alumnotd@pc2damA-28:~$ cd AndroidStudioProjects/  
alumnotd@pc2damA-28:~/AndroidStudioProjects$ ll  
total 4  
drwxr-x--- 3 alumnotd alumnotd 4096 sep 21 13:27 HolaMundo
```

2. Abre un terminal en la carpeta y ejecuta:

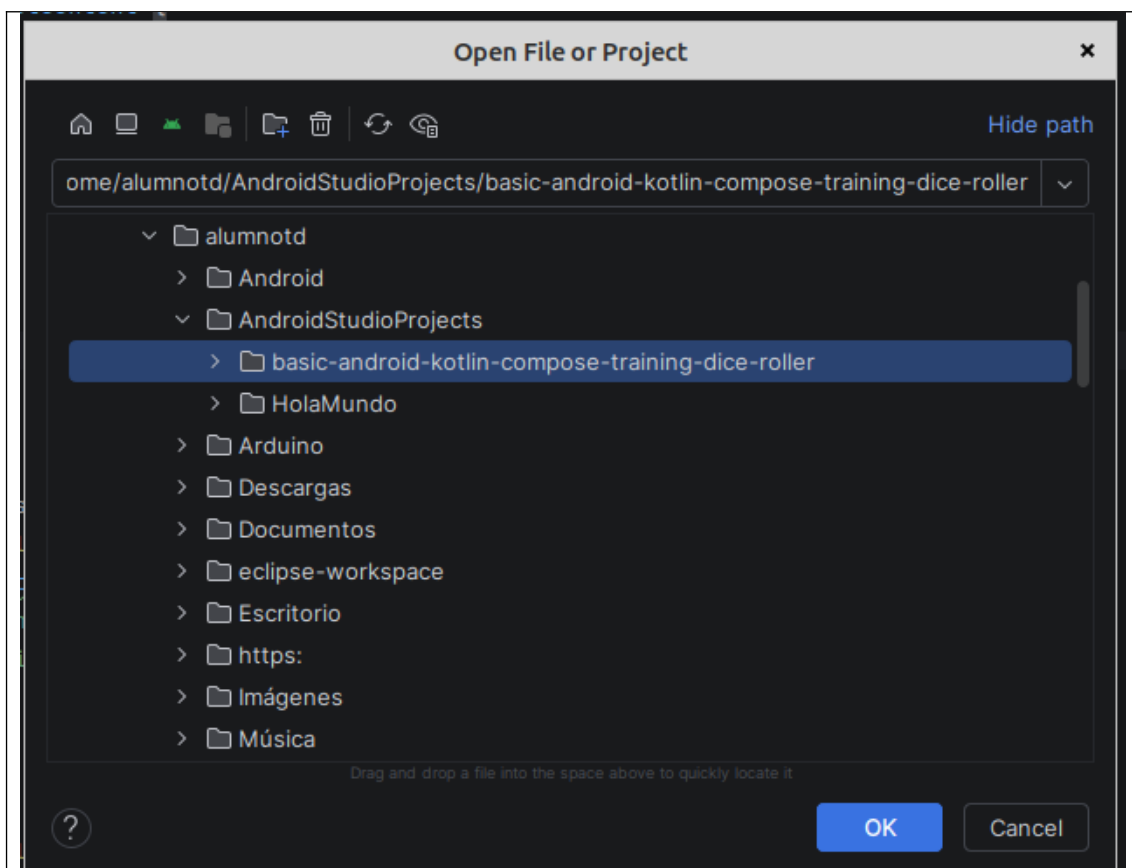
git clone <https://github.com/google-developer-training/basic-android-kotlin-compose-training-dice-roller.git>

Nota: es posible que git te pida credenciales.

```
alumnotd@pc2damA-28:~/AndroidStudioProjects$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-dice-roller.git  
Clonando en 'basic-android-kotlin-compose-training-dice-roller'...  
remote: Enumerating objects: 283, done.  
remote: Counting objects: 100% (83/83), done.  
remote: Compressing objects: 100% (35/35), done.  
remote: Total 283 (delta 60), reused 48 (delta 46), pack-reused 200 (from 1)  
Recibiendo objetos: 100% (283/283), 197.20 KiB | 2.29 MiB/s, listo.  
Resolviendo deltas: 100% (123/123), listo.
```

PASO 2 (0,5 puntos)

Abre el proyecto con Androd Studio, seleccionando la carpeta que se ha creado (basic-android-kotlin-compose-training-dice-roller).



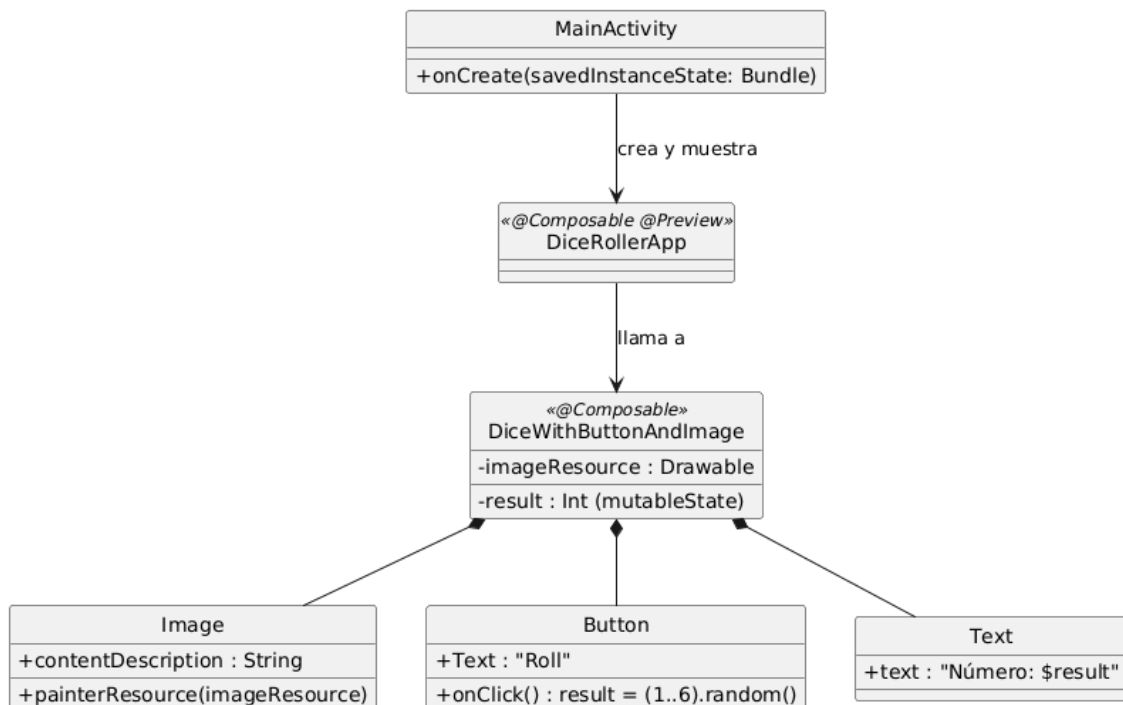
PASO 3 (0 puntos)

a) Revisa la composición de la app:

Componente	Tipo	Función
MainActivity	Activity	Inicializa la UI y aplica tema
DiceRollerApp()	Composable	Organiza layout general y centra contenido
DiceWithButtonAndImage()	Composable	Contiene lógica, imagen, número y botón

result	State	Guarda el valor actual del dado
Image	Composable	Muestra el dado según result
Text	Composable	Muestra el número obtenido
Button	Composable	Cambia result aleatoriamente

b) Fíjate como La Clase DiceWithButtonAndImage es una composición (como las que te explicaron en programación de primer curso):



c) Revisa las carpetas:

app/java/com.example.diceroller

- Contiene el **código Kotlin** de la aplicación.
- Clase principal MainActivity.kt → punto de entrada de la app.

app/res/

- Archivos XML que definen la **interfaz gráfica (UI)**.

app/res/drawable/

- Recursos gráficos (iconos de los dados).

app/res/values/

- Archivos XML con **strings, colores, estilos**.
- Ejemplo: strings.xml → textos de la app.

AndroidManifest.xml

- Configuración de la app (actividad principal, permisos).

build.gradle (app y proyecto)

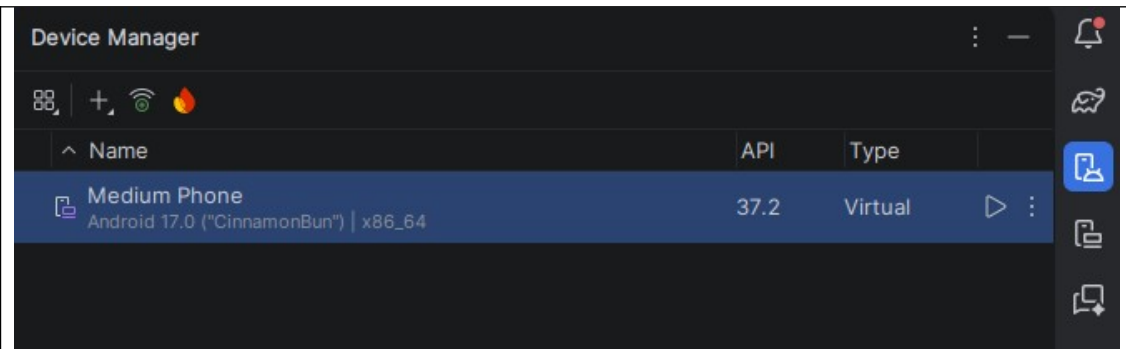
- Configuración del compilador, dependencias y plugins.

Nota: algunas carpetas pueden cambiar al actualizarse el proyecto. Por ejemplo, en lugar de app/java, aparecerá app/Kotlin+java.

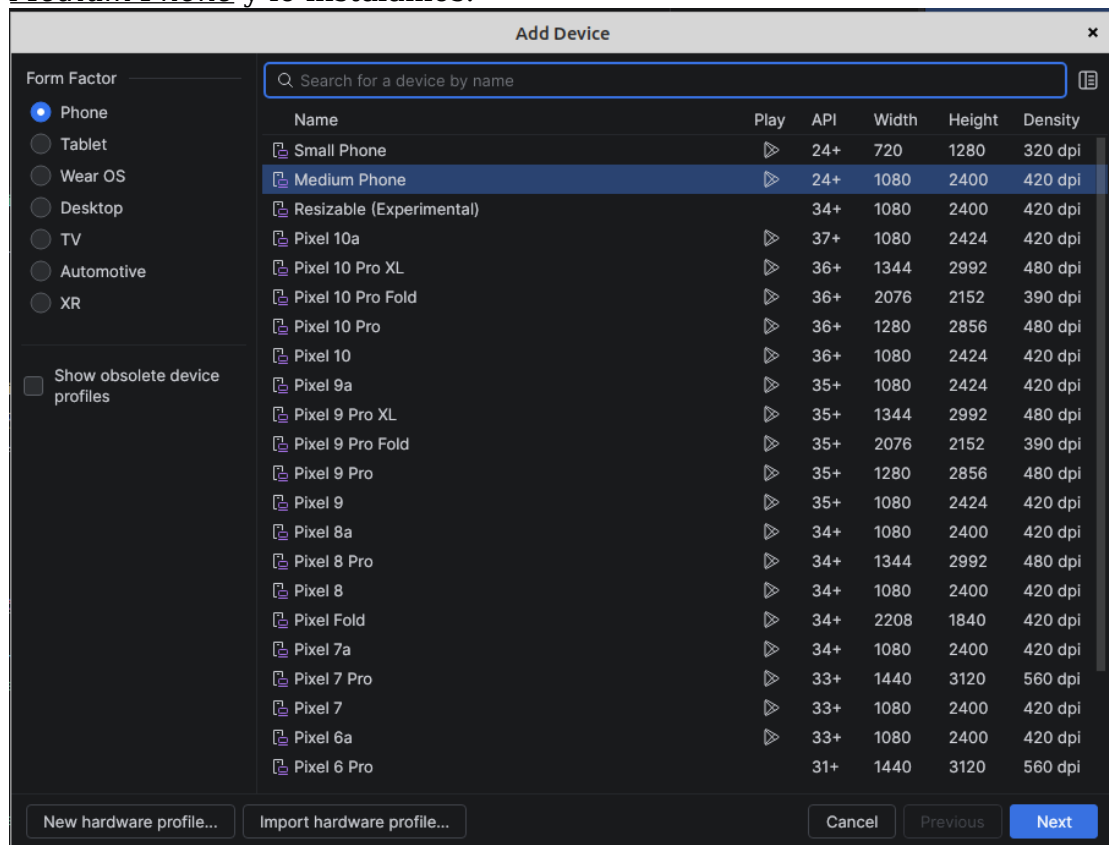
PASO 4 (0,5 puntos)

Ejecuta y prueba la aplicación en el emulador.

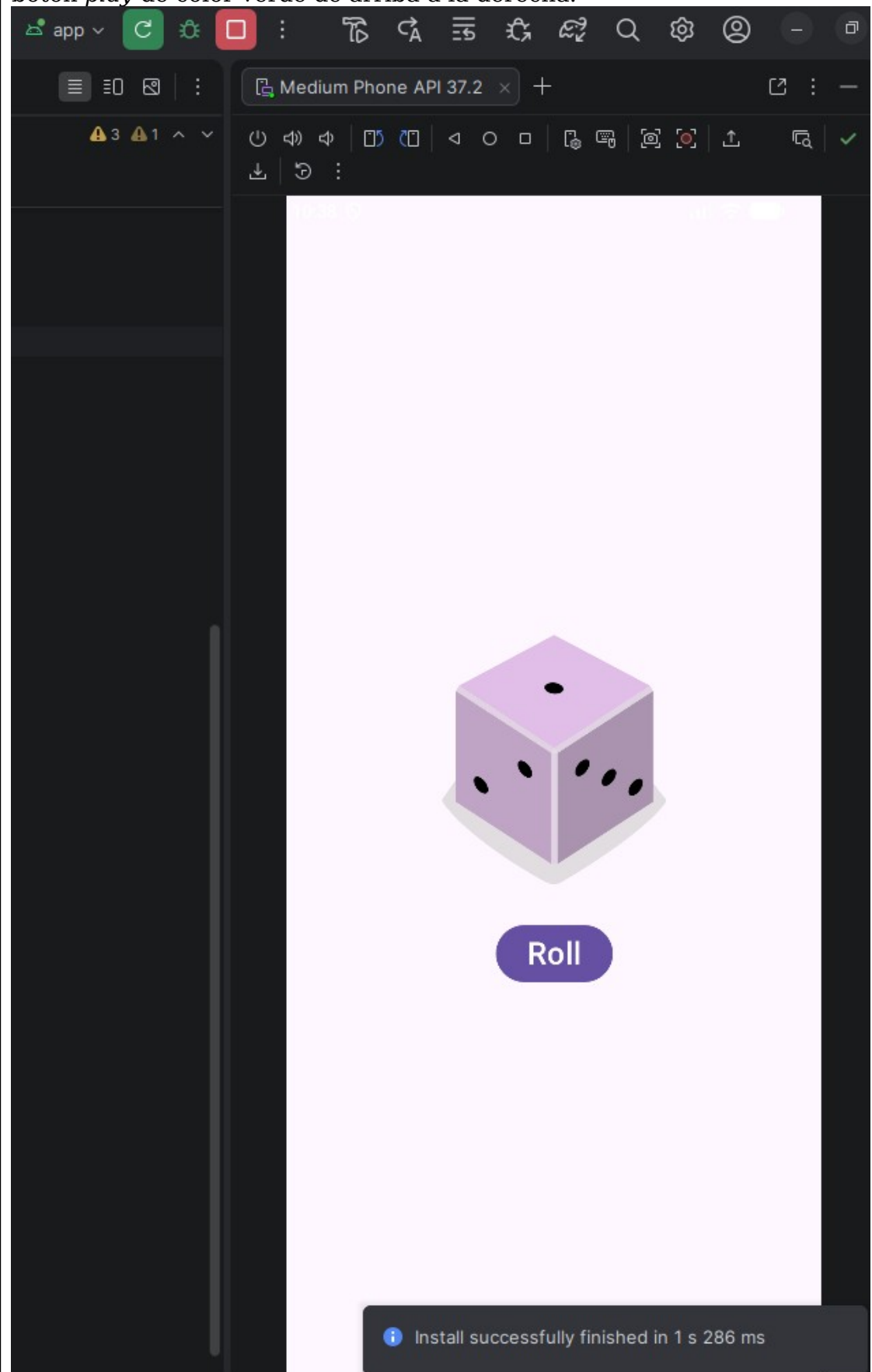
En la parte derecha del IDE buscamos el símbolo que tiene un icono de un móvil y la mascota de Android, clicamos en él y revisamos que tengamos instalado un emulador de un teléfono:



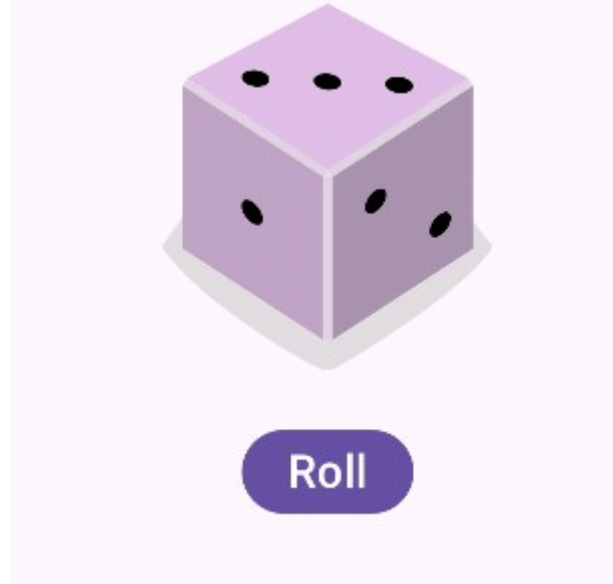
Si no tenemos uno instalado hacemos clic en el símbolo más (+), indicamos que queremos crear un nuevo dispositivo virtual, buscamos *Medium Phone* y lo instalamos.



Una vez lo tenemos instalado ejecutamos la aplicación clicando en el botón *play* de color verde de arriba a la derecha.



Para probar la aplicación clicamos en el botón que dice *Roll*



PASO 5 (2 puntos)

- a) ¿Si instalaras tu aplicación en el móvil con qué nombre aparecería en el escritorio? (Pista: el nombre se encuentra en una etiqueta XML).

Se llamaría *Dice roller*

```
<application
    android:icon="@mipmap/ic_launcher"
    android:label="Dice Roller"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:theme="@style/Theme.DiceRoller"
    tools:targetApi="33">
    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:theme="@style/Theme.DiceRoller">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

- b) Identifica en el código la parte que genera el número aleatorio. Modifica el código para que sólo genere un valor entre 1-4. Pruébalo.

```

Button(
    onClick = { result = (1 ≤ .. ≤ 4).random() },
) {
    Text(text = stringResource(id = R.string.roll), fontSize = 24.sp)
}

```

c) Localiza cómo se actualiza la **imagen del dado**:

```

@Composable
fun DiceWithButtonAndImage(modifier: Modifier = Modifier) {
    var result by remember { mutableStateOf(value = 1) }
    val imageResource = when(result) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        else -> R.drawable.dice_6
    }

    Column(modifier = modifier, horizontalAlignment = Alignment.CenterHorizontally) {
        Image(painter = painterResource(id = imageResource), contentDescription = result.toString())

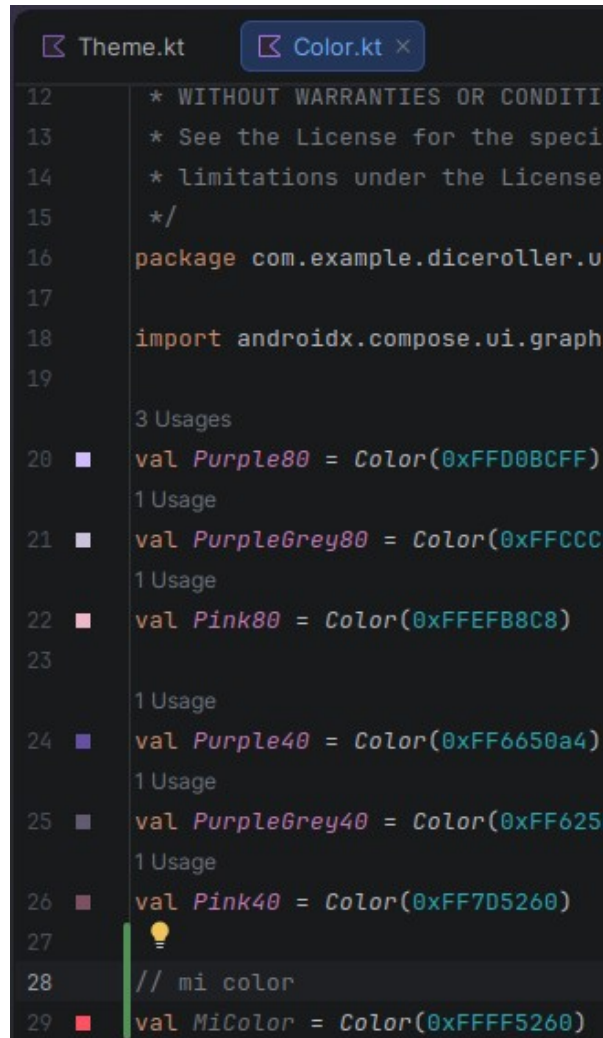
        Button(
            onClick = { result = (1 ≤ .. ≤ 4).random() },
        ) {
            Text(text = stringResource(id = R.string.roll), fontSize = 24.sp)
        }
    }
}

```

PASO 6 (2 puntos)

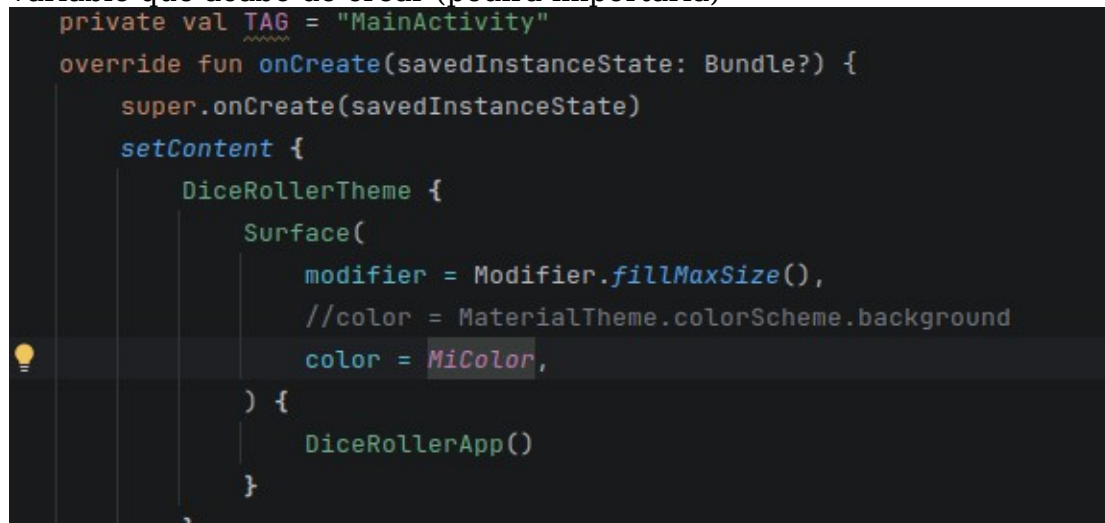
- a) Modifica el color de fondo y el estilo del botón. Pruébalo.

Creo una variable *MiColor* en el fichero *Color.kt*



```
12  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either
13  * See the License for the specific language governing
14  * limitations under the License.
15  */
16  package com.example.diceroller.ui
17
18  import androidx.compose.ui.graphics.Color
19
20  3 Usages
21  val Purple80 = Color(0xFFD0BCFF)
22  1 Usage
23  val PurpleGrey80 = Color(0xFFCCC2CF)
24  1 Usage
25  val Pink80 = Color(0xFFE78F6C)
26  1 Usage
27  val Purple40 = Color(0xFF6650a4)
28  1 Usage
29  val PurpleGrey40 = Color(0xFF625971)
30  1 Usage
31  val Pink40 = Color(0xFF7D5260)
32
33  // mi color
34  val MiColor = Color(0xFFFF5260)
```

Luego indico que el color que quiero usar como color de fondo es esa variable que acabo de crear (pedirá importarla)



```
private val TAG = "MainActivity"
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContent {
        DiceRollerTheme {
            Surface(
                modifier = Modifier.fillMaxSize(),
                //color = MaterialTheme.colorScheme.background
                color = MiColor,
            ) {
                DiceRollerApp()
            }
        }
    }
}
```



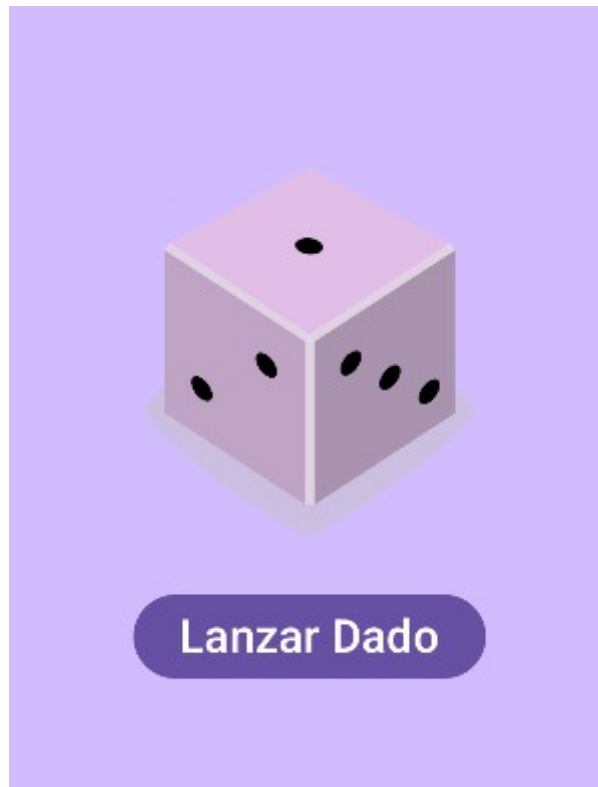
Lanzar Dado

Número: 1

b) Cambia el texto del botón ("Roll" → "Lanzar dado"). Pruébalo.

Haciendo *ctr + click* en *R.string.roll* me lleva a donde está declarada la variable, por lo que puedo cambiar el valor de esta:

```
<resources>
  <string name="app_name">Dice Roller</string>
  <string name="roll">Roll</string>
</resources>
```

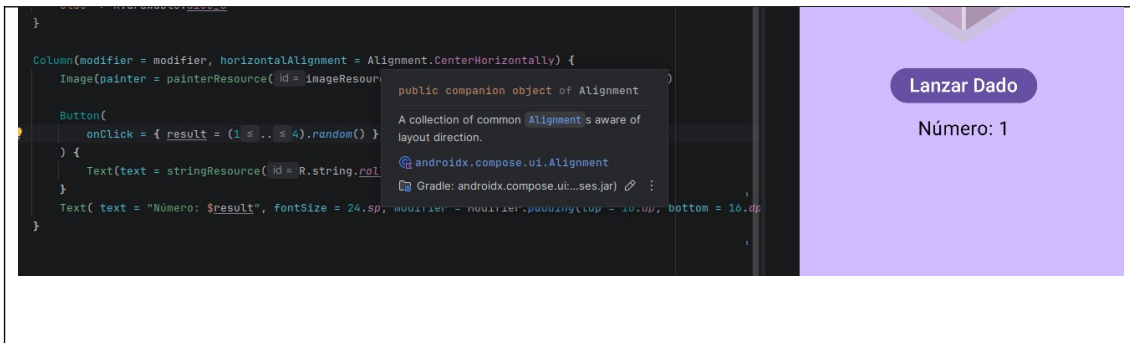


c) Además de la imagen, mostrar en pantalla un `TextView` con el valor obtenido en el dado. Pista: En Jetpack Compose, no se usa `TextView` como en la interfaz clásica de Android; en su lugar, se usa `Text`, que cumple la misma función.

Simplemente añade un `Text` debajo de la imagen para mostrarlo.

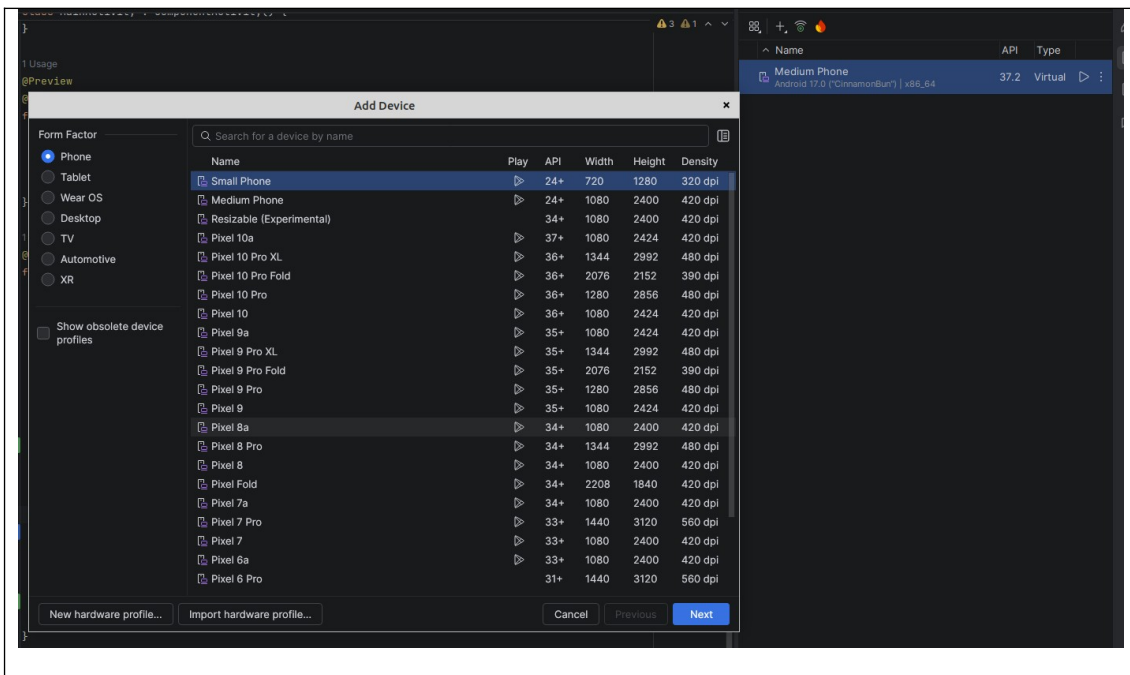
```
Text( text = "Número: $result", fontSize = 24.sp, modifier =
Modifier.padding(top = 16.dp, bottom = 16.dp) )
```

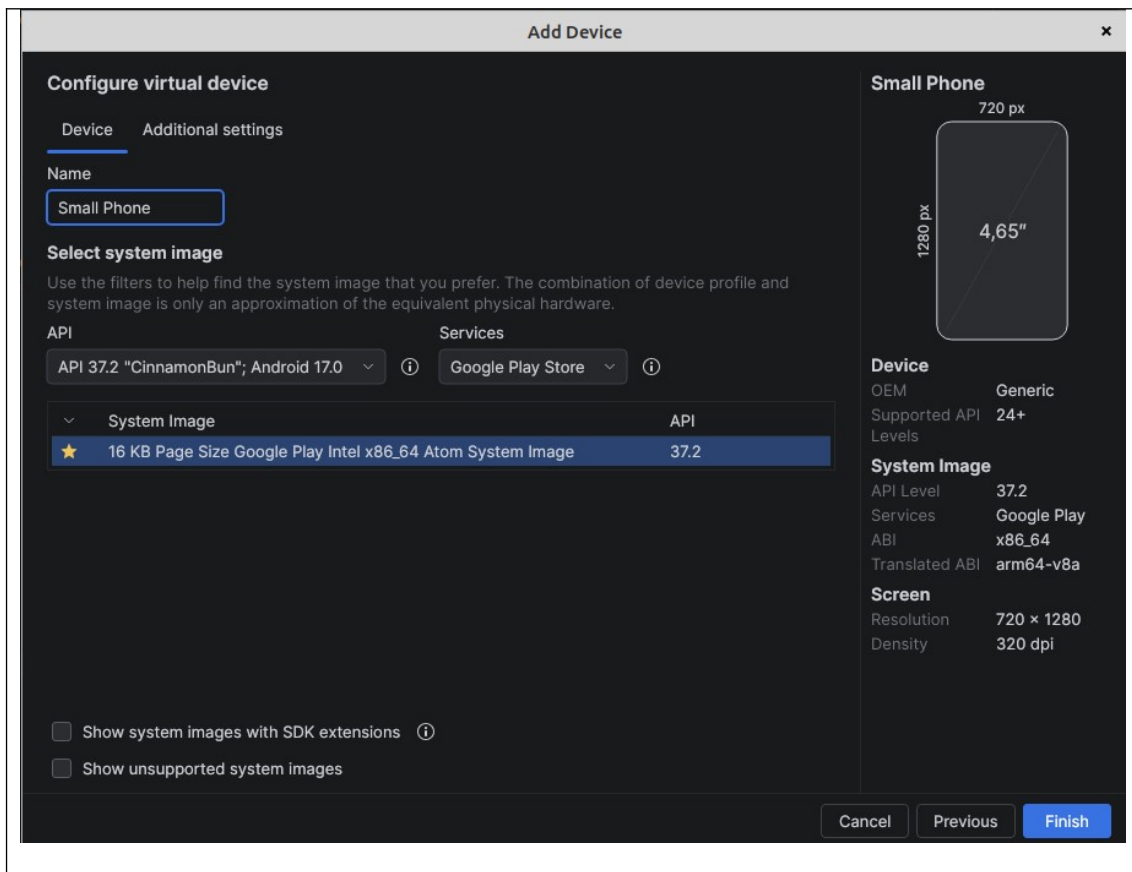
Después, Pruébalo.



PASO 7 (2 puntos)

a) Ejecútalo en otro dispositivo virtual.

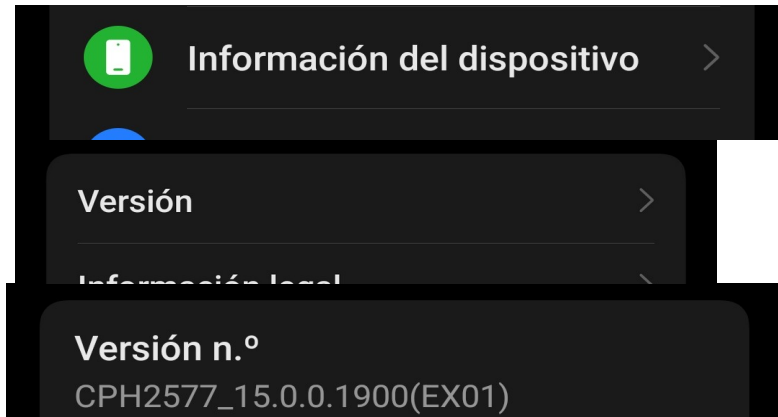




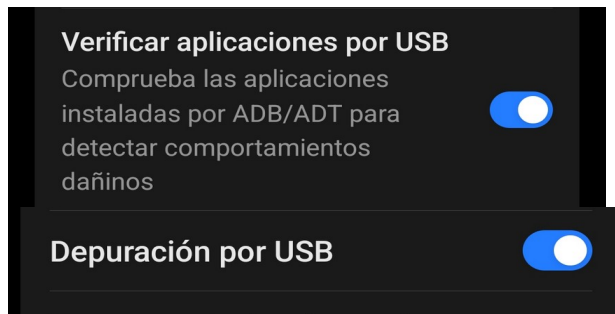
b) Instálalo en tu móvil y pruébalo.

Si no dispones de Android o no puedes hacerlo por algún motivo, indícalo. Después explica y muestra detalladamente los pasos que habría que seguir para hacerlo (aunque no lo hagas). Siempre que sea posible, muestra capturas de pantalla de tu móvil (dónde se activa el modo desarrollador, muestra cómo lo activas...)

Abrimos los ajustes, presionamos en información del dispositivo, presionamos en Versión, presionamos 7 veces en el número de versión y nos pedirá el método de desbloqueo.



Ahora tenemos que activar la depuración por USB, para esto buscamos las opciones de desarrollador y activamos la depuración y la instalación vía USB.



Cuando lo tengamos activado Tendremos que conectarlo al ordenador mediante un cable USB. Una vez conectado tendremos que seleccionarlo en Android Estudio y ejecutar la aplicación. Cuando lo hallamos hecho se habrá instalado el programa en nuestro teléfono móvil.

PASO 8 (2 puntos)

Vamos a ver implementados los eventos del ciclo de vida de una app de Android.

- Cambia Actividad (MainActivity) principal para que sobrescriba todos y cada uno de los eventos. Para ello:

- Declara la siguiente variable en la clase MainActivity:

`private val TAG = "MainActivity"`

```
class MainActivity : AppCompatActivity() {  
    6 Usages  
    private val TAG = "MainActivity"
```

- Añade justo después (dentro de la clase MainActivity) la siguiente sobreescritura de métodos por cada evento del ciclo de vida:

```
override fun onStart() {  
    super.onStart()  
    Log.d(TAG, "onStart")  
}
```

```
override fun onResume() {  
    super.onResume()  
    Log.d(TAG, "onResume")  
}
```

```
override fun onPause() {  
    super.onPause()  
    Log.d(TAG, "onPause")  
}
```

```
override fun onStop() {  
    super.onStop()  
    Log.d(TAG, "onStop")  
}
```

```
override fun onRestart() {  
    super.onRestart()  
    Log.d(TAG, "onRestart")  
}
```

```
override fun onDestroy() {  
    super.onDestroy()  
    Log.d(TAG, "onDestroy")  
}
```

```

class MainActivity : ComponentActivity() {
    6 Usages
    private val TAG = "MainActivity"
    override fun onCreate(savedInstanceState: Bundle?) {...}
    override fun onStart() {
        super.onStart()
        Log.d(TAG, msg = "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, msg = "onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, msg = "onPause")
    }

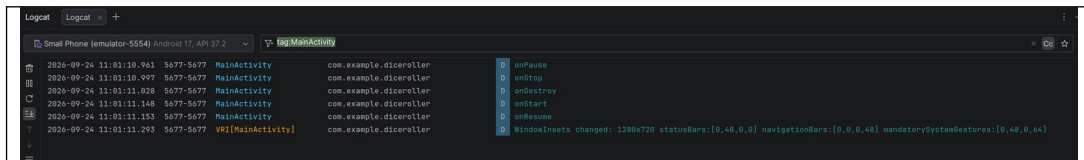
    override fun onStop() {
        super.onStop()
        Log.d(TAG, msg = "onStop")
    }

    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, msg = "onRestart")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, msg = "onDestroy")
    }
}

```

- Abre LogCat para ver los mensajes de MainActivity, y filtra por tag:MainActivity. Ejecuta la aplicación y pruébala. También gira el dispositivo virtual. Después revisa los logs y busca los mensajes que hemos incluido en los estados.



Clicamos en el botón con forma de gato del panel de la izquierda, en el filtro ponemos *tag:MainActivity* para filtrar por el tag que hemos creado, cuando interacturemos con el giro de la pantalla se actualizarán los logs.

Normas de entrega:

La entrega deberá ser a través de Aula Virtual. Si todavía no se dispone de usuario de EducaMadrid, se permitirá excepcionalmente la entrega por email. Se entregará un documento Word que contendrá el enunciado y las soluciones, con el siguiente nombre de fichero:

<GRUPO>_<APELLIDO1>_<NOMBRE>.<extensión de fichero>

En cada parte delimitada con < y > se tendrá que sustituir según el valor correspondiente. Todo irá en mayúsculas (salvo la extensión) y sin tildes. Por ejemplo, si una persona llamada Ángel Pérez fuera a clase de 2DAM, sería:

2DAM_PEREZ_ANGEL.docx

Evaluación:

Cada apartado indica su puntuación. Se valorará especialmente la profundidad y claridad de las explicaciones. Solo se obtendrá la máxima nota en un apartado si está perfectamente, y paso a paso, explicado y documentado con capturas de pantalla.

Además, se tendrán en cuenta los siguientes criterios de calificación adicionales:

- Fecha de entrega: Si la entrega no se realiza a tiempo, la calificación de la práctica será de 0 puntos.
- Adecuación al nombre del fichero: si no cumple el formato de nombre del fichero especificado, se restará 0,5 puntos.
- Entrega adecuada: si no se ha entregado el zip con el proyecto, o no se ha entregado este documento relleno, la calificación de la práctica será de 0 puntos. Hay que entregar ambos. Además, por cada captura de pantalla que no se haya incluido, o paso omitido o no explicado correctamente, se restará la mitad del valor del apartado.
- Si se detecta copia o que la solución ha sido generada por inteligencia artificial o alguna herramienta similar no autorizada, la calificación será de 0 (tanto para el que ha copiado como para el que se ha dejado copiar, en el caso de copia). No compartas tu solución con nadie. Si tienes dudas sobre si se puede utilizar alguna herramienta, consúltalo con tu profesor antes de usarla. El profesor puede realizar una revisión oral de la práctica con los alumnos que considere que deben explicar su código para corroborar su autoría.